

Foundations of Data Science for Biologists

Class Orientation and Introduction to R

BIO 724D

2026-AUG-25

Instructors: Greg Wray and Paul Magwene

Orientation to BIO724D

What is BIO 724D?

A class that introduces how to work with biological data

Practical skills for wrangling, processing, filtering, and visualizing data

Best practices for analysis, interpretation, reproducibility, and reusability of data

Assumes no prior programming experience

A class that builds community

Fostering discussions with other grad students, post, staff, and faculty

Drawing on data sets generated by people in the Biology department

Not

An introduction to statistics (though we will often use statistical methods)

An introduction to computer science (though it might feel like it occasionally)

Course structure

Lecture / lab: Mondays 3:05-5:05pm, 145 BioSci

Semi-flipped format: complete reading / video assignments *before* class

Class sessions mostly active learning with some traditional lecture

Hands-on component mostly follow-along using structured notebooks

Data lunch: Thursdays 12:00-1:00pm, 144 BioSci

Most session are presentations by grad students, postdocs, staff, and faculty

Some sessions are familiarization with tools, resources, projects, etc.



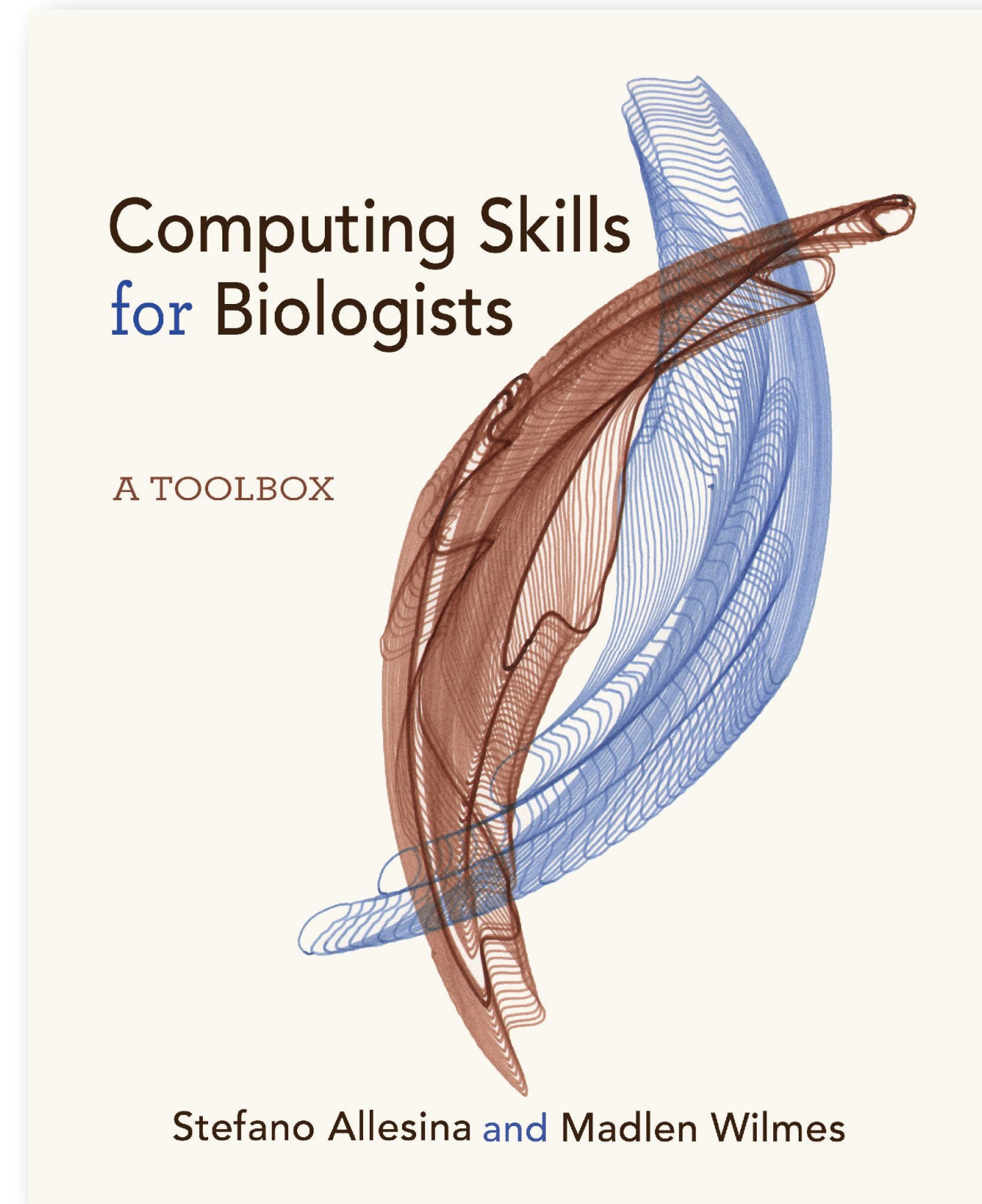
be respectful



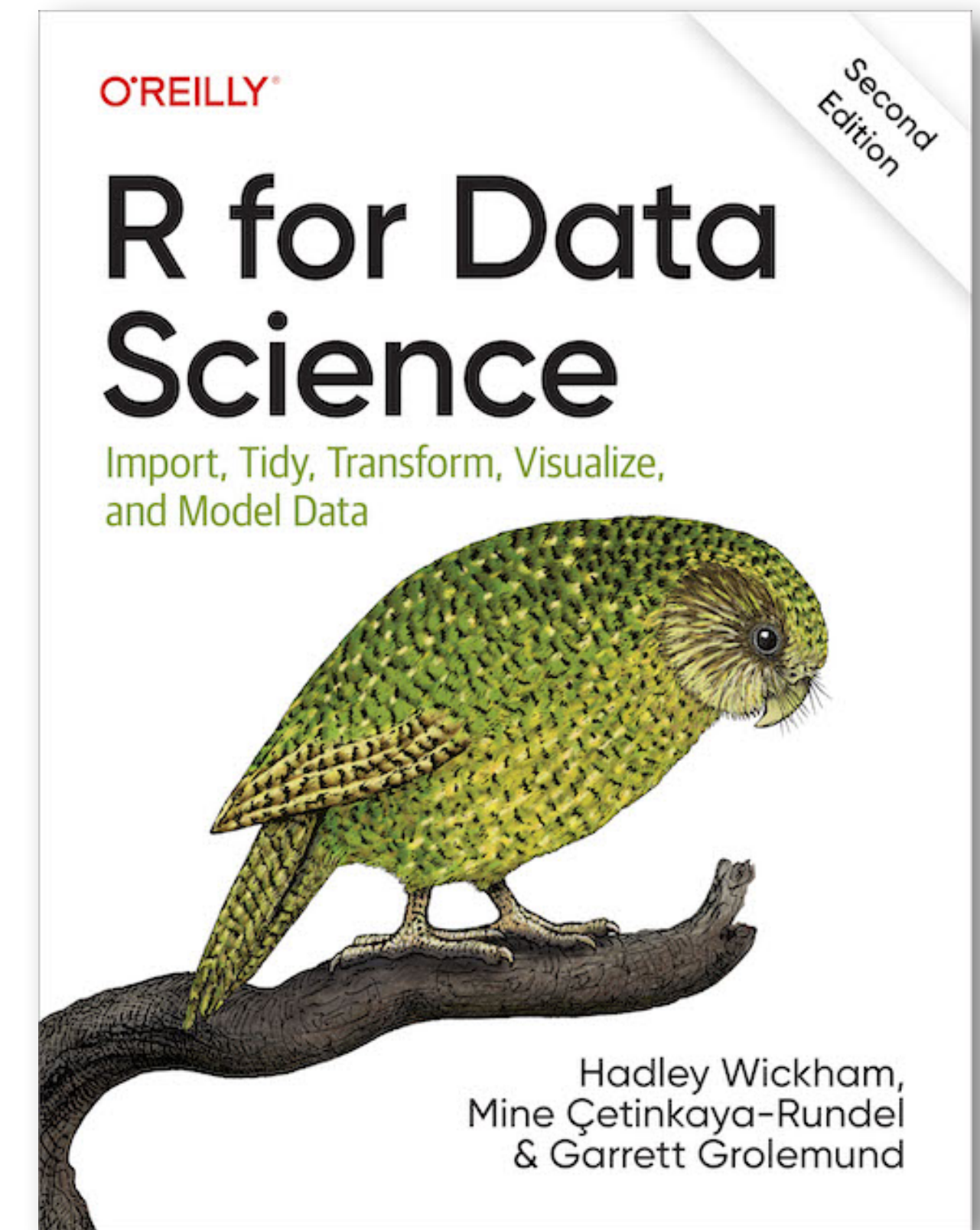
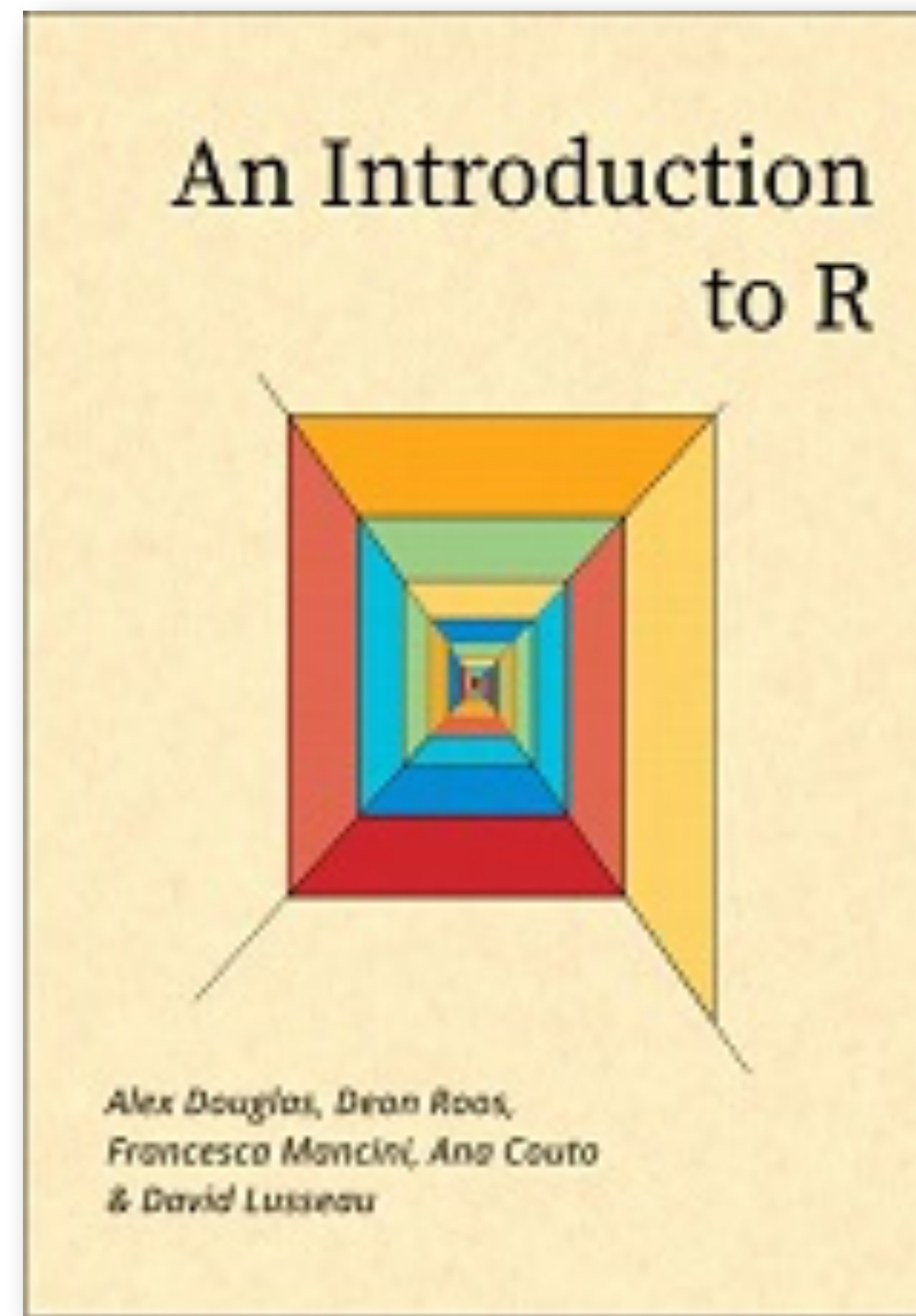
be polite

Course materials

Throughout the year:



For the R portion of the course:



*All are available electronically for **free**: see the course wiki*

Practicalities

Wiki: *the hub for most things related to the course*

- Schedule and information about course logistics, grading, and policies

- Links to preparation materials: readings and videos

- Links to Tuesday materials: slide decks, data sets, notebooks, extra resources

- Assignments: weekly problem sets and project rubric

GitHub: *where you turn in assignments*

- Upload files into your folder under the “Assignments” tab

- We will cover how to set up your GitHub repository on Thursday

Duke Hub: *where your grades are recorded*

- Check assignment credit and scores

Expectations and keys to success

Adhere to the Duke Community Standard

Attend every class session and actively engage

Come to Tuesday sessions prepared

Submit homework and projects on time

Stay on top of the material and ask questions

Familiarize yourself with the information on the course wiki

Assignments

Problem sets (90%)

Posted by class time each Monday; due the following Sunday at midnight

Graded for completion: code must run or include an explanation

Data lunch reflections (5%)

Short description of new idea/method/application you learned

Turn in with your weekly problem set; graded for completion

Learning notebook (5%)

Markdown file of notes, code snippets, links, etc.; due Dec 6

Projects

Rubric posted on Oct 1, due Dec 6

Counts as 2 problem sets

Seeking help

Seek help when you are stuck!

Developer's resources: documentation and code snippets from website or repo

Traditional resources: textbooks, Google, YouTube

On-line forums: StackOverflow, Python.org, Posit Community Forum, and others

AI resources: ChatGPT, Claude, BingGPT, GPT4/5, etc.

Human resources: classmates, friends, lab-mates, other grad students

When turning in your homework, cite your sources and explain how they helped

E.g., "Code for removing empty rows modified from a post on StackOverflow"

E.g., "Use of Itertools library for permutation suggested by Claude"

For humans: use generic attributions (e.g., "a classmate", "a lab member")

For AI: name the model and include the prompt you used

Code that isn't working

There ~~may~~ will be occasions when you can't get your code to work properly: **no problem!!**

You will get **full** credit if you:

- Explain how you tried to fix the problem

- Identify specifically what is and is not working correctly

- Turn your homework in on time

You will **not** get full credit if you:

- Turn your assignment in late without prior arrangement

- Neglect to reveal and explain what isn't working properly (see above)

Test your code carefully and be transparent about any issues you can't fix!

What can you expect from BIO 724D?

You will learn concepts and skills that will:

- Be useful throughout your research career

- Save you *lots* of time and frustration

- Reduce errors in your work

- Make it easier for you to collaborate

This comes at a cost

- You will need to practice, practice, and put in effort

- You may feel frustrated at times, but persevere and it will be worth it!

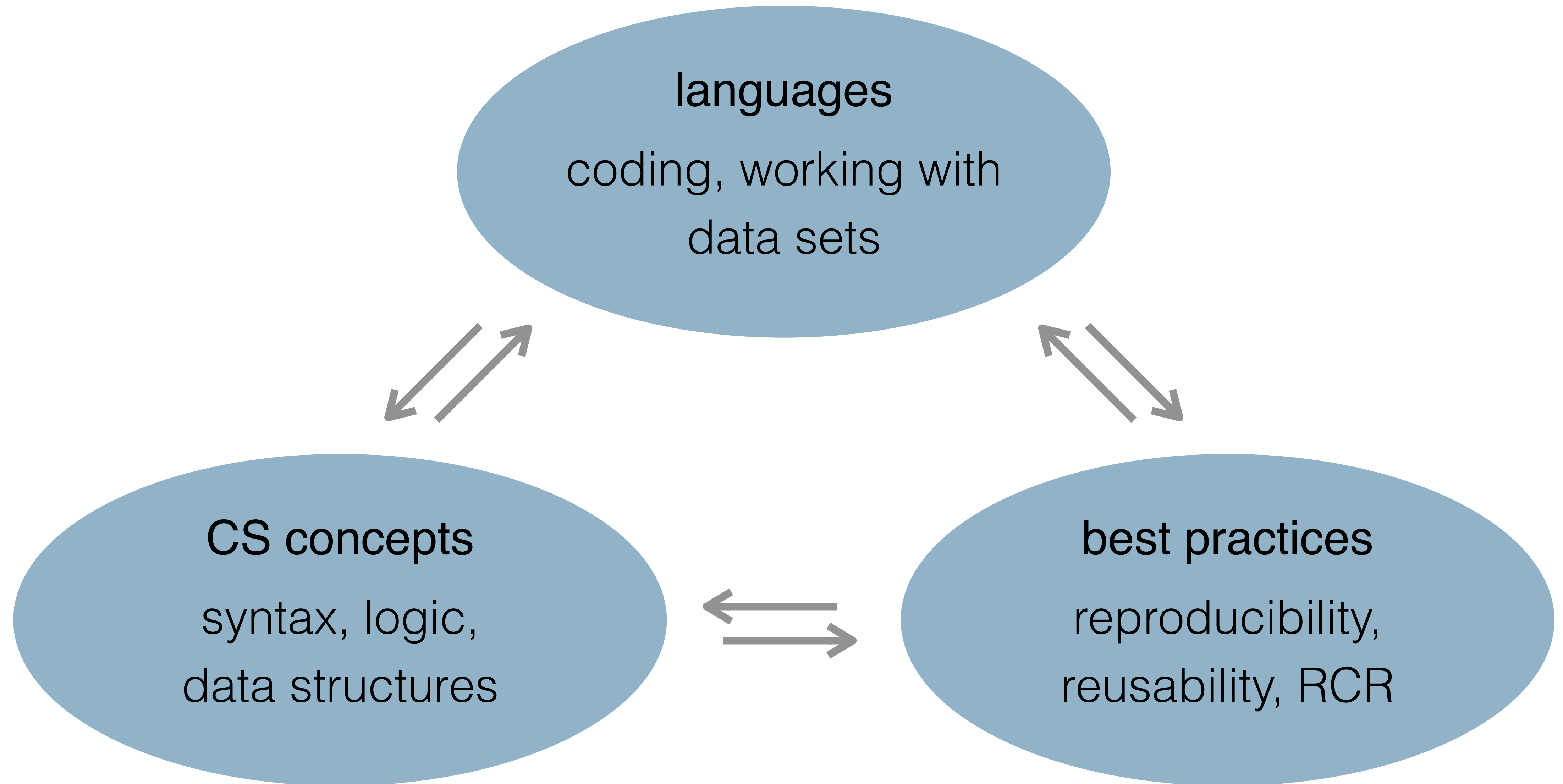
We will strive to make this process as smooth as possible

- Maintain a safe, respectful, and constructive learning environment

- Encourage questions and foster curiosity (no “dumb” questions)

- Provide support and resources to help you learn, thrive, and grow

Learning objectives



Learning objectives: the fine print

How to work with data:

- Format data so that it is easy to work with

- Reproducibly identify missing data, improperly formatted data, and outliers

How to write code that:

- Works as intended

- Makes sense to future-you and to others

- Saves time and effort, reduces errors, and improves reproducibility

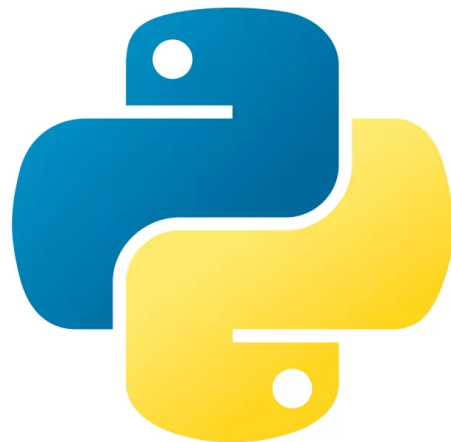
- Can be re-purposed for other tasks, even unrelated ones

How to produce visuals that:

- Reveal trends, organization, and anomalies within complex data sets

- Portray your data with clarity, accuracy, and integrity

Languages we will work with

	<i>strengths</i>	<i>weaknesses</i>
	Designed for statistics and visualization Good for numerical modeling Widely used in biological research	Size of data sets limited by RAM Not optimized for performance
	Designed for relational databases Extraordinarily powerful in this role Dominant database environment	Nearly useless outside its niche Procedural programming is awkward
	Nearly universal computing environment Extremely powerful, uniquely capable Most future-proof way to code	Terse syntax, not easily readable Designed to work primarily with text
	General-purpose high-level language Readable code, consistent syntax Among most widely used languages	“Second-best at everything” Somewhat optimized for performance

Preview of topics

session	topics
1 - 11	R : importing, transforming, querying, and displaying data
12 - 13	SQL : designing and querying relational databases
	<i>winter break</i>
14 - 19	Unix : working with files at the command line; regular expressions
20 - 26	Python : general-purpose programming

Introduction to R and R Studio

Why learn R?

Designed specifically for data analysis, modeling, and visualization

Designed by statisticians for statisticians: rigorous, tested, and respected

Free, open-source, non-proprietary, cross-platform, actively maintained

Encourages best practices in reproducibility, visualization, and sharing

Huge set of tested libraries for specialized tasks

Used by biologists in many areas of study, familiar to many collaborators

Numerous libraries specific for biological research (e.g., GIS, modeling, genomics)

What is the difference between R and RStudio?



programming language

does the computing
“engine”



programming environment

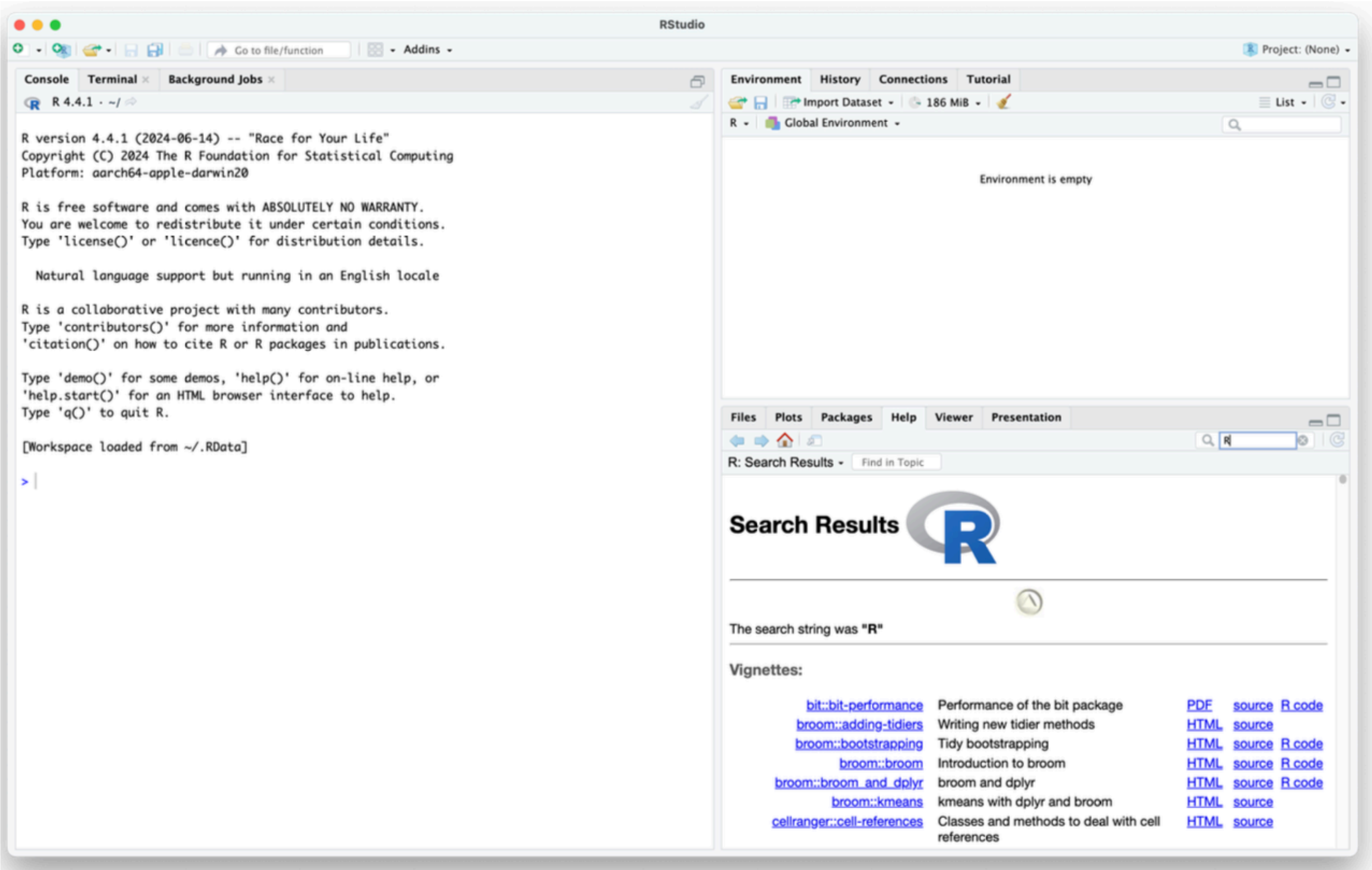
provides an interface
“dashboard”



RStudio

R version →

prompt →



Typing at the console

If you enter an expression at the console, R will return the result:

```
> 6 * 7                                # multiplication
[1] 42
> sqrt(9)                              # using a function to find the square root
[1] 3
```

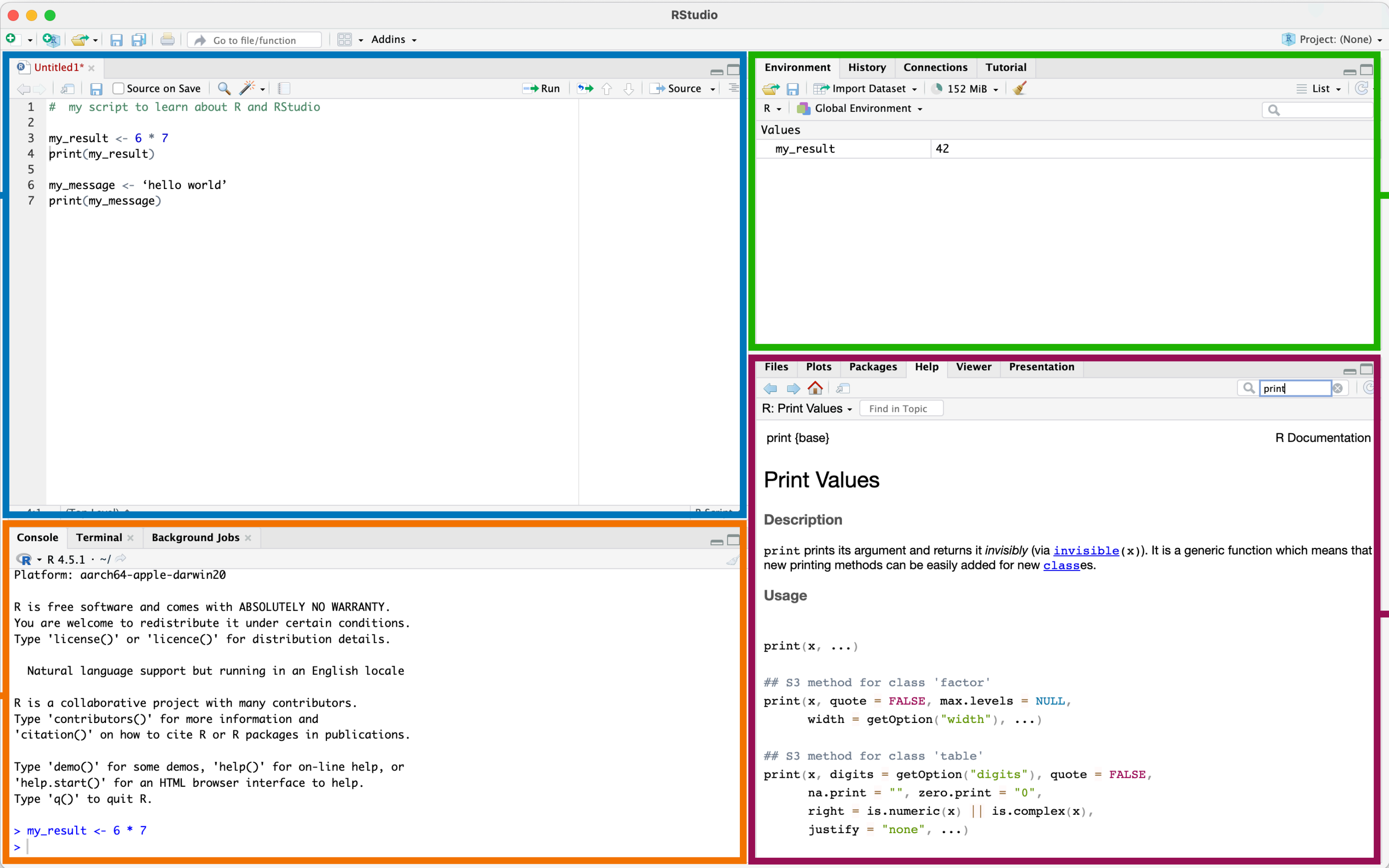
To store a result, assign it to a variable name:

```
> my_result <- 6 * 7                  # <- is the assignment operator
> print(my_result)                    # displays the value stored in my_result
[1] 42
```

When you quit R Studio, any stored values will be lost! The solution is to store code.

Quadrants of the RStudio interface

code
editor



Store your code in a script

Let's write some code we want to save:

```
# my script to learn about R and RStudio

my_result <- 6 * 7
print(my_result)

my_message <- 'hello world'
print(my_message)
```

Save the file to preserve your work — you can retrieve it later by opening the file

Scripts are very useful for: saving time, record-keeping, reproducibility, reusability

Some tips for writing scripts

Use comments at the beginning of the script to indicate overall purpose, date, author, etc.

Use comments throughout to explain what your code does

Use descriptive names for variables

Use whitespace for readability (blank lines and extra spaces are generally ignored)

Save time!

Use **tab** to autocomplete names of functions and variables

Use arrow keys to select the desired value when there are multiple options

Use keyboard shortcuts:

<- **opt** + **-** on Mac, **alt** + **-** on Windows (note: minus key)

Full list under Help menu, Keyboard Shortcuts Help

